Arquitectura de computadoras

Act. ALU



CUCEI

Clave: IL365 NRC: 200204 Seccion: D03 Calendario: 2026B

Ariel Rojas Garcia

221930873

1. INTRODUCCIÓN

1.1 La ALU y su papel en el procesador

La Unidad Aritmético-Lógica (ALU, por sus siglas en inglés) es el componente del procesador que ejecuta las operaciones aritméticas (suma, resta) y lógicas (AND, OR, NOR, comparaciones) sobre los datos que le entregan los registros. Recibe dos operandos, una señal de control que indica qué operación realizar y produce un resultado, además de banderas de estado; en este diseño, la bandera Zero (ZF) indica si el resultado fue cero, dato que el procesador usa, por ejemplo, para decidir saltos condicionales.

1.2 Historia breve de la ALU

El concepto de unidad aritmética como órgano independiente del computador aparece en el documento "First Draft of a Report on the EDVAC" (1945), de John von Neumann, donde la arquitectura se divide en unidad aritmética, unidad de control, memoria y dispositivos de entrada/salida. En las primeras computadoras la ALU se construía con tubos de vacío y después con transistores discretos, ocupando gabinetes completos.

Con los circuitos integrados llegaron las ALU como un solo chip: un ejemplo clásico es el 74181 de Texas Instruments (1970), una ALU de 4 bits capaz de realizar 16 operaciones aritméticas y 16 lógicas, que podía encadenarse para formar ALU más anchas (enfoque "bit-slice"). En 1971, el Intel 4004 integró la ALU, los registros y el control en un solo chip, dando origen al microprocesador. Desde entonces el ancho de palabra creció de 4 a 64 bits y los procesadores modernos incluyen varias ALU que trabajan en paralelo (diseños superescalares).

1.3 Tipos de ALU

Las ALU pueden clasificarse de varias maneras. Según el tipo de dato que procesan:

- **ALU entera:** opera con números enteros (suma, resta, lógica, corrimientos). Es la que se implementa en esta actividad.
- **Unidad de punto flotante (FPU):** realiza operaciones con números reales representados en punto flotante (IEEE 754), como suma, multiplicación y división.
- **Unidad de generación de direcciones (AGU):** ALU especializada en calcular direcciones de memoria.
- **ALU vectoriales / SIMD:** aplican la misma operación a varios datos al mismo tiempo.

Según su organización física pueden ser **ALU de un solo chip de ancho completo**, como las de los procesadores actuales, o **ALU en rebanadas de bits (bit-slice)**, donde varias ALU pequeñas se combinan para lograr un ancho mayor. Finalmente, según su implementación interna, la suma

puede realizarse con sumadores de acarreo en cascada (ripple-carry), más simples pero lentos, o con sumadores de acarreo anticipado (carry-lookahead), más rápidos pero con más hardware.

1.4 Diferencias entre la ALU de una CPU y la de una GPU

La ALU de una CPU es una unidad (o unas pocas, en diseños superescalares) capaz de ejecutar operaciones aritméticas y lógicas con baja latencia, dentro de núcleos complejos con predicción de saltos, ejecución fuera de orden y grandes cachés. Su objetivo es resolver rápido cada instrucción individual.

Una GPU, en cambio, integra miles de ALU simples (núcleos CUDA en NVIDIA) organizadas en bloques que comparten una unidad de control y memoria local, y ejecutan la misma instrucción sobre muchos datos a la vez (modelo SIMT). Cada ALU de GPU es más simple y más lenta por operación, pero el conjunto ofrece un rendimiento agregado (throughput) muy superior en tareas masivamente paralelas como gráficos, álgebra matricial y redes neuronales.

1.5 Asignación por bloqueo (=) y no bloqueo (<=) en Verilog

En Verilog, dentro de un bloque procedural (always o initial) existen dos operadores de asignación:

- **Asignación por bloqueo (=, blocking):** se ejecuta en el orden en que está escrita; la siguiente sentencia espera a que la actual termine y el valor se actualiza de inmediato. Se comporta como el código secuencial de un lenguaje de programación.
- **Asignación por no bloqueo (<=, non-blocking):** el lado derecho se evalúa en ese momento, pero la actualización de la variable se programa para el final del paso de simulación. Así, todas las asignaciones de un mismo bloque "ocurren al mismo tiempo", lo cual modela el comportamiento de registros que cambian en el mismo flanco de reloj.

La regla de buena práctica ampliamente aceptada (por ejemplo, en los trabajos de Cummings sobre estilos de codificación) es usar = en bloques combinacionales (always @(*)) y <= en bloques secuenciales (always @(posedge clk)).

En el módulo de esta actividad se utilizó <= dentro de un bloque combinacional. En este caso particular el resultado es correcto, porque cada rama del case asigna una sola variable (ALUout) una única vez y no hay dependencias entre asignaciones; sin embargo, la práctica recomendada para lógica combinacional sería usar =.

2. OBJETIVO

Objetivo general

Diseñar, describir en Verilog y verificar mediante simulación un módulo ALU de 32 bits, en estilo comportamental, que realice las operaciones de suma, resta, AND, OR, NOR y SLT sobre dos operandos de 32 bits seleccionadas mediante una señal de control de 4 bits, y que genere la bandera Zero.

Objetivos particulares

- Describir el módulo MIPSALU con entradas A y B de 32 bits, selector ALUctrl de 4 bits, salida ALUout de 32 bits y salida ZF.
- Implementar la selección de la operación mediante una sentencia case dentro de un bloque always combinacional.
- Generar la bandera ZF cuando el resultado de la operación sea igual a cero.
- Construir un testbench que ejercite cada operación con al menos tres pares de operandos distintos.
- Visualizar los resultados de la simulación en formato decimal y validarlos contra el cálculo esperado.
- Investigar y documentar la historia, los tipos de ALU, sus diferencias entre CPU y GPU, y la diferencia entre asignación por bloqueo y no bloqueo.

3. DESARROLLO.

3.1 Especificación de las operaciones

La señal ALUctrl de 4 bits selecciona la operación. Los códigos utilizados son los de la ALU del procesador MIPS (Patterson y Hennessy). Cualquier otro valor de ALUctrl produce ALUout = 0.

ALUctrl (decimal)	ALUctrl (binario)	Operación	Expresión en Verilog
0	0000	AND	A & B
1	0001	OR	A | B

2	0010	Suma	A + B
6	0110	Resta	A - B
7	0111	SLT (set on less than)	A < B ? 1 : 0
12	1100	NOR	~(A | B)
Otro	—	Sin operación	0

Tabla 1. Operaciones de la ALU y su código de selección.

3.2 Módulo de la ALU (MIPSALU.v)

El módulo se describe de forma comportamental. Los puntos principales son:

- **Puertos:** A y B son buses de entrada de 32 bits y ALUctrl es el selector de 4 bits. ALUout se declara como output reg porque se le asigna dentro de un bloque always; una señal asignada en un bloque procedural debe ser de tipo reg.
- **Bloque always @(*):** la lista de sensibilidad automática hace que el bloque se reevalúe cada vez que cambia cualquiera de sus entradas (A, B o ALUctrl), por lo que el circuito es combinacional.
- **Sentencia case:** selecciona la operación según ALUctrl, y el default asegura que todas las combinaciones no utilizadas produzcan cero, evitando que ALUout conserve un valor anterior (lo cual inferiría un latch).
- **Bandera ZF:** se genera con una asignación continua (assign) que compara ALUout con cero, por lo que se actualiza automáticamente con cada resultado.
- **Aritmética sin signo:** A y B no se declararon con signo, por lo que la suma, la resta y la comparación de SLT se realizan en aritmética de 32 bits sin signo, con desbordamiento circular (wrap-around).

3.3 Testbench (MIPSALU_TB.v)

El testbench declara los operandos A_tb y B_tb y el selector ALUctrl_tb como reg, y las salidas ALUout_tb y ZF_tb como wire, e instancia la ALU conectando cada puerto por nombre. Dentro de un bloque initial se aplican tres pares de operandos; para cada par se recorren las seis operaciones (ALUctrl = 0, 1, 2, 6, 7 y 12), manteniendo cada combinación 100 ns. En total se ejercitan 18 casos (3 por operación), con una duración de simulación de 1800 ns, al término de la cual se ejecuta $stop.

Conjunto	A	B	Propósito
1	121	31	Caso ordinario con valores pequeños
2	4294967295	1	Valor máximo de 32 bits: prueba el desbordamiento y los bits en 1
3	0	1	Operando en cero: prueba la bandera ZF y el resultado negativo en la resta

Tabla 2. Pares de operandos utilizados en el testbench.

3.4 Resultados esperados

La siguiente tabla muestra el resultado esperado de cada caso, calculado manualmente y contrastado con una simulación del diseño. Todos los valores están en formato decimal, como pide la actividad.

A	B	Operación	ALUout	ZF

121	31	AND	25	0
121	31	OR	127	0
121	31	Suma	152	0
121	31	Resta	90	0
121	31	SLT	0	1
121	31	NOR	4294967168	0
4294967295	1	AND	1	0
4294967295	1	OR	4294967295	0
4294967295	1	Suma	0	1
4294967295	1	Resta	4294967294	0
4294967295	1	SLT	0	1
4294967295	1	NOR	0	1

0	1	AND	0	1
0	1	OR	1	0
0	1	Suma	1	0
0	1	Resta	4294967295	0
0	1	SLT	1	0
0	1	NOR	4294967294	0

Tabla 3. Resultados esperados de las 18 pruebas del testbench.

Como verificación manual del primer conjunto (A = 121, B = 31): en binario, 121 = 1111001 y 31 = 0011111, por lo que AND = 0011001 (25) y OR = 1111111 (127); la suma es 152 y la resta 90; 121 no es menor que 31, así que SLT = 0; y NOR es el complemento de OR sobre 32 bits: $2^{32} - 1 - 127 = 4294967168$.

3.5 Análisis de casos especiales

- **Desbordamiento en la suma:** con A = 4294967295 y B = 1, la suma real es 2^{32}, que no cabe en 32 bits; el resultado se trunca a 0 y por eso ZF = 1. La ALU no cuenta con una salida de acarreo, por lo que este desbordamiento no se señala.
- **Resta con resultado negativo:** con A = 0 y B = 1, la resta da −1, que en complemento a dos de 32 bits equivale a 4294967295 (todos los bits en 1). Al mostrarse el resultado como decimal sin signo aparece este valor.
- **SLT sin signo:** con A = 4294967295 y B = 1 el resultado es 0, porque A y B se interpretan sin signo y A es mayor que B. Si se interpretaran con signo (complemento a dos), A valdría −1 y el resultado sería 1. Esto debe tenerse en cuenta al decidir cómo se usará la ALU en un procesador completo.

- **Bandera ZF:** se activa en los casos cuyo resultado es cero: SLT con A ≥ B (cuando el resultado es 0), la suma con desbordamiento, el NOR con A = 4294967295 y el AND con A = 0.

3.6 Resultados y validación en ModelSim

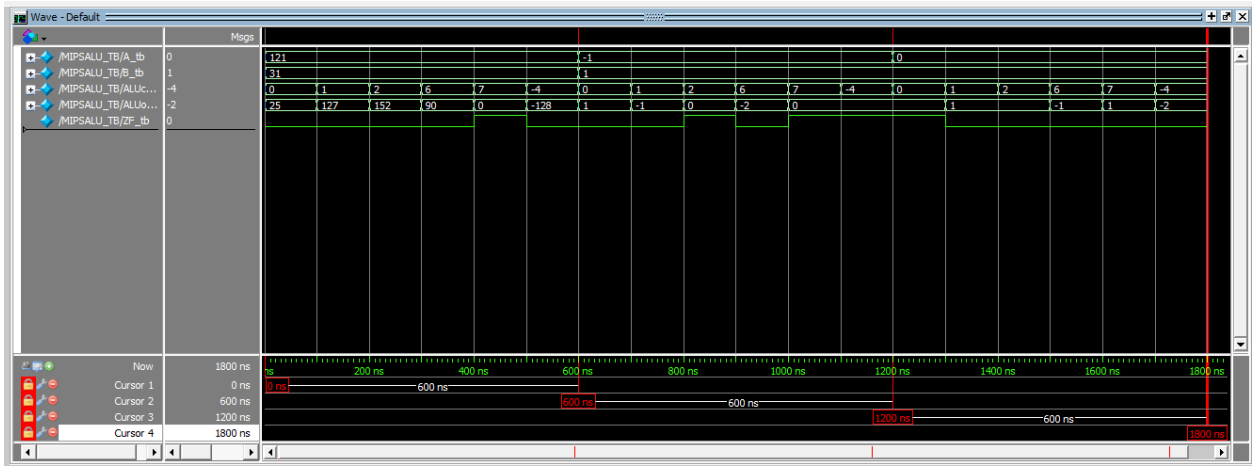


Figura 1. Ventana Wave de resultados obtenidos en la simulación de MIPSALU_TB.v.

Los valores observados en la forma de onda coinciden con los de la Tabla 3: cada cambio de ALUctrl (cada 100 ns) produce el resultado correspondiente en ALUout y el valor de ZF esperado.

4. CONCLUSIÓN

Esta actividad me permitió construir el primer bloque aritmético de un procesador: una ALU de 32 bits que, con un solo selector de 4 bits, decide entre seis operaciones. Describirla de forma comportamental, con un always @(*) y una sentencia case, resultó mucho más compacto que armar cada operación con compuertas o sumadores, y dejó claro que el trabajo de la herramienta de síntesis es convertir esa descripción en hardware.

La simulación también me mostró detalles que no son evidentes al leer el código: el desbordamiento circular de la suma (4294967295 + 1 = 0), la representación de los números negativos en complemento a dos al restar 0 − 1, y el hecho de que, al no declarar los operandos con signo, la operación SLT compara valores sin signo. Además, la investigación sobre asignación por bloqueo y no bloqueo me hizo notar que, aunque el código funciona usando <= en un bloque combinacional, lo recomendable es usar = en ese tipo de lógica y reservar <= para la lógica secuencial. Por último, la bandera ZF, generada con una simple asignación continua, es un ejemplo de cómo las señales de estado de la ALU se derivan directamente del resultado y serán útiles más adelante para las instrucciones de salto.

5. REFERENCIAS

- Patterson, D. A., & Hennessy, J. L. Computer Organization and Design: The Hardware/Software Interface. Morgan Kaufmann.
- Hennessy, J. L., & Patterson, D. A. Computer Architecture: A Quantitative Approach. Morgan Kaufmann.
- Mano, M. M., & Ciletti, M. D. Diseño Digital. Pearson Educación.
- Palnitkar, S. Verilog HDL: A Guide to Digital Design and Synthesis. Prentice Hall.
- Cummings, C. E. (2000). Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill! SNUG (Synopsys Users Group).
- IEEE. IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364). IEEE Computer Society.
- von Neumann, J. (1945). First Draft of a Report on the EDVAC.
- NVIDIA Corporation. CUDA C++ Programming Guide — arquitectura de streaming multiprocessors.